Space Defender: A 2D Particle-Based Shooter Game

Submitted By

Student Name	Student ID
Raiyan Jahan	0242310005101599
Md Rizwan Karim Lam	0242310005101608
Tantu Chakrabarty	0242310005101426

LAB PROJECT REPORT

This report is Presented in Partial Fulfillment of the course

CSE412: Computer Graphics Lab in the Department of Computer Science and Engineering

DAFFODIL INTERNATIONAL UNIVERSITY

Dhaka, Bangladesh

21/04/2026

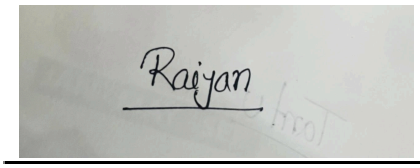
DECLARATION

We hereby declare that this lab project has been done by us under the supervision of **Fahiba Farhin, Assistant Professor,** Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as lab projects.

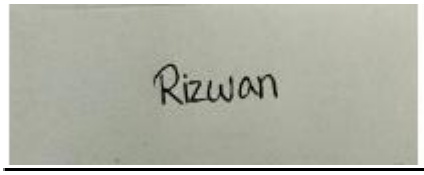
Submitted To:

Fahiba Farhin
Assistant Professor
Department of Computer Science and Engineering
Daffodil International University

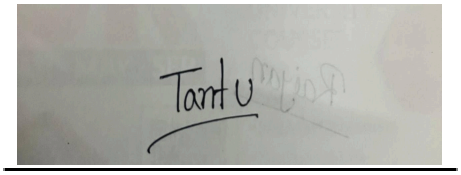
Submitted by



Raiyan Jahan
ID:0242310005101599
Dept. of CSE, DIU



Md Rizwan Karim Lam
ID:0242310005101608
Dept. of CSE, DIU



Tantu Chakrabarty
ID:0242310005101426
Dept. of CSE, DIU

COURSE & PROGRAM OUTCOME

The following courses have course outcomes as follows:

Table 1: Course Outcome Statements

CO1	Design and implement fundamental computer graphics primitives and algorithms to construct 2D graphical scenes for real-life visualization problems having complex engineering attributes.
CO_2	Apply appropriate OpenGL programming techniques, geometric transformations, and graphics algorithms to solve complex real-world visualization problems using modern engineering tools.
CO_3	Perform effectively as an individual and as a member of a diverse team to design, document, and implement computer graphics–based projects.
CO4	Create and present a complete computer graphics project by explaining complex engineering activities through effective reports, demonstrations, and presentations.

Mapping Course Outcomes (COs) with the Teaching-Learning and Assessment Strategy

The mapping justification of this table is provided in sections **4.2.1**, **4.2.2,** and **4.2.3**.

Table 2: Mapping of CO, PO, Blooms, KP, and C

CO	**POs**	**Teaching Learning Activities**	**Assessment Strategy**	**Learning Domains**	**Knowledge Profile**	**Complex Engineering Problem**	**Complex Engineering Activities**
CO1	PO1	TLA1 (Lab exercises)	Lab performance, Lab report	C2	K1, K3	EP1, EP4	
CO_2	PO5	TLA2 (Algorithm implementation)	Lab performance, Lab report	C3, C4	K6	EP1, EP4, EP3	
CO_3	PO9	TLA3 (Group project)	Lab project	P3, A2			
CO4	PO10	TLA4 (Presentation & Viva)	Project & Viva	C6, P3, A2			EA1, EA3

Table of Content

DECLARATION..... 2
COURSE & PROGRAM OUTCOME..... 3
Introduction..... 5
1.1 Introduction..... 5
1.2 Motivation..... 5
1.3 Objectives..... 5
1.4 Feasibility Study..... 6
1.5 Gap Analysis..... 6
1.6 Project Outcome..... 7
System Architecture/Scene Description & Architecture..... 7
2.1 Requirement Analysis & Design Specification..... 7
2.1.1 Scene Overview..... 7
2.1.2 Object Components..... 8
Implementation and Results..... 10
3.1 Implementation..... 10
Proposed Model:..... 10
3.2 Output:..... 11
3.3 Discussion..... 12
Engineering Standards and Mapping..... 13
4.1 Impact on Society, Environment and Sustainability..... 13
4.1.1 Impact on Life..... 13
4.1.2 Impact on Society & Environment..... 13
4.1.3 Ethical Aspects..... 13
4.1.4 Sustainability Plan..... 13
4.2 Complex Engineering Problem..... 14
4.2.1 Mapping of Program Outcome..... 14
4.2.2 Complex Problem Solving..... 14
Conclusion..... 16
5.1 Summary..... 16
5.2 Limitation..... 16
5.3 Future Work..... 16
References..... 17

Chapter 1

Introduction

This chapter gives an overview of the Space Defender computer graphics project, including its objectives, scope, and importance. It explains the project theme, game concept, and goals achieved through graphical modeling, real-time animation, and interactive gameplay.

1.1 Introduction

Computer graphics is a field of computer science that deals with creating and displaying visual content using a computer. It is used in many areas such as game development, simulation, education, and digital art. OpenGL (Open Graphics Library) is one of the most widely used tools for rendering 2D and 3D graphics. It provides a set of functions that allow programmers to draw shapes, apply colors, and create interactive animations in real time.

This project is a 2D real-time arcade shooter game named Space Defender, built using OpenGL and the FreeGLUT library in C. The game places the player in control of a spaceship at the bottom of a 700x700 pixel starfield window. Asteroids of varying sizes and speeds fall from the top of the screen, and the player must shoot them with bullets before they escape off the bottom edge. The project demonstrates the use of graphics primitives, rendering algorithms, 2D transformations, a particle effect system, and a complete game-state machine within a single interactive application.

1.2 Motivation

The motivation behind this project was to create a visually engaging and interactive experience using the graphics concepts learned in the Computer Graphics Lab course. Space-themed shooter games are a classic genre of computer gaming that everyone is familiar with, making it a compelling and relatable theme. The team wanted to go beyond simple static or animated drawings and build a fully playable game with real-time input, collision detection, and dynamic visual effects. The concept of a player-controlled ship shooting down named asteroids was chosen because it allows the effective demonstration of multiple graphics techniques simultaneously: particle systems, real-time transformation, state management, and modular rendering.

1.3 Objectives

The main objectives of this project are:

- To implement graphics algorithms such as the Midpoint Circle Algorithm and Bresenham Line Algorithm for drawing scene elements, including asteroids, ship components, and bullet tips.
- To apply 2D transformations, specifically translation, to animate the starfield background, falling asteroids, player bullets, and particle effects.

.

- Create a complete interactive arcade game using OpenGL and FreeGLUT, including a splash screen, gameplay, and game-over states.
- To implement a real-time particle system generating engine thruster trails and asteroid explosion effects
- To design a modular and well-structured OpenGL program with separate functions for each entity type and game subsystem.
- The project aims to showcase real-time rendering by utilizing the GLUT timer function, achieving a frame rate of approximately 60 frames per second.

1.4 Feasibility Study

Many existing OpenGL-based student projects focus on simple static scenes or single moving objects. Computer graphics courses commonly develop projects such as village sceneries and basic car animations. However, a complete real-time interactive game that combines a parallax starfield, a player-controlled entity, multiple dynamically spawned enemies with collision detection, an explosion particle system, and a heads-up display with score tracking is significantly more advanced. Our project is feasible because all required components can be built using standard OpenGL primitives, trigonometric mathematics, and the freely available FreeGLUT library, which works across different platforms. The C programming language provides the necessary performance and control for real-time game logic.

1.5 Gap Analysis

Most similar student graphics projects either render a static scene or animate only one or two objects along a predetermined path. They rarely implement interactive input handling, dynamic entity spawning, real-time collision detection, or particle effects. Additionally, most existing examples do not include game-state management with splash screens and game-over screens, nor do they feature difficulty progression. This project fills that gap by building a complete interactive application where multiple systems operate simultaneously: the player moves in real time, bullets travel upward, asteroids spawn at random positions and speeds, particles are emitted continuously, and the game scales in difficulty as the score increases. Each subsystem is isolated in its set of functions, making the codebase readable and extensible.

1.6 Project Outcome

The final output of this project is a 700x700 pixel interactive game window. The game begins with a styled splash screen displaying the university name, project title, team members, and controls. Once The player presses space, and the game begins. The player flies a cyan spaceship at the bottom of a scrolling starfield, shooting bullet projectiles upward. Asteroids of randomized sizes (radius 18 to 28 pixels) fall from the top at increasing speeds. Each asteroid carries a randomly assigned name label from the team's name pool. Destroying an asteroid scores 100 points and triggers a multi-particle explosion burst with a beep sound. If an asteroid passes the bottom edge, the player loses a life. The game ends when all three lives are exhausted, displaying a game-over screen with the final score. The player can press R to restart at any time.

.

Chapter 2

Scene Description & Architecture

This chapter describes the overall game design and architecture of the Space Defender project. It outlines the main graphical components, their interactions, game logic, and the technologies used to build the game.

2.1 Requirement Analysis & Design Specification

2.1.1 Scene Overview

The project renders a space shooter game inside a 700 X 700 pixel OpenGL display. The coordinate system starts from the bottom-left corner (0,0) and extends to (700,700) using orthographic projection set by gluOrtho2D(0, WIN_W, 0, WIN_H). The game window is divided into functional layers:

- HUD Layer (y = 670 to 700): Contains score (left), level (center), and lives (right) displays rendered with GLUT bitmap fonts.
- Game Space (y = 40 to 670) is the active play area where asteroids spawn, bullets travel, and the player operates. The scrolling star parallax background fills this region.
- Player Zone (y = 60): Fixed vertical position of the player ship. The ship moves only along the x-axis within the range x = 30 to 670.
- Control Bar (y = 0 to 40): Static text at the bottom showing the control scheme.

2.1.2 Object Components

The major visual objects in the game are listed below with their geometric properties:

Object	Primitive Used	Description
Starfield	GL_TRIANGLE_FAN (circles)	80 parallax scrolling stars of varying brightness and speed
Player Ship	GL_TRIANGLES	Cyan triangular hull, wing extensions, cockpit circle, dual engine glow, name label
Asteroids	GL_TRIANGLE_FAN (circles)	Dynamic radius 18-28px; layered circles for rock texture, crater, and outline; name label below

.

Bullets	GL_QUADS + circle tip	Cyan 6x18 pixel elongated rectangle with glowing white circular tip
Explosion Particles	GL_TRIANGLE_FAN (circles)	Up to 40 particles per explosion in yellow, orange, and white; fade out with gravity
Engine Trail	GL_TRIANGLE_FAN (circles)	Two blue particle streams emitted from the ship's engine nozzles every 3 frames
HUD	GLUT Bitmap Fonts	Score, level, and lives rendered with GLUT_BITMAP_HELVETICA_18 and GLUT_BITMAP_HELVETICA_12
Splash / Game Over	GL_QUADS + text	Multi-section splash with university branding, team info, controls, and start prompt; semi-transparent game-over overlay

2.2 Use of Modern Tools

- **OpenGL / FreeGLUT:** Used for rendering all 2D visual elements, managing the display window, handling user input callbacks, and driving the animation loop via glutTimerFunc.
- **C Programming Language:** The core language used to write the graphics algorithms, entity structs, game logic, drawing functions, and animation update loop.
- **GLUT Timer Function:** Used to control the game loop, calling the update and redisplay functions every 16 milliseconds (approximately 60 FPS).
- **Windows Beep API / POSIX Terminal Bell:** Cross-platform sound effect triggered on asteroid destruction using conditional compilation (#ifdef _WIN32).
-
- **Git / GitHub:** Used for version control and collaboration among team members during development.

.

2.3 Algorithm Used:

The following graphics algorithms are applied in this project:

Midpoint Circle Algorithm:

This algorithm is used to draw all circular shapes in the scene. In the code, the drawCircle() function draws a filled circle by computing 30 equally spaced points around the center using trigonometric calculations (cosf and sinf), which are the computational equivalents of the midpoint circle algorithm. The algorithm is used for the player's cockpit glass, asteroid bodies, engine glow circles, bullet tips, and all particle effect discs.

```
void drawCircle(float cx, float cy, float r) {
    int i;
    glBegin(GL_TRIANGLE_FAN);
    glVertex2f(cx, cy);
    for (i = 0; i <= 30; i++) {
        float a = i * 2.0f * PI / 30. 0f;
        glVertex2f(cx + cosf(a) * r, cy + sinf(a) * r);
    }
    glEnd();
}
```

Bresenham Line Algorithm:

This algorithm is used conceptually in drawing all straight-line structures. In the code, GL_LINES is used to draw the horizontal divider lines in the splash screen, which follows the principle of efficient integer-based straight-line pixel rendering across the screen.

Scan-Line Fill (GL_POLYGON/GL_QUADS/GL_TRIANGLES):

OpenGL's built-in polygon rasterization acts as a scan-line fill algorithm when drawing all filled shapes. GL_TRIANGLES is used for the player ship's hull and wings. The HUD background rectangles and bullet bodies both use GL_QUADS. GL_TRIANGLE_FAN is used for all circles. This technique fills every solid shape in the game.

2.4 2D Transformations Applied

The primary 2D transformation used throughout this project is translation. Multiple game elements are translated during each frame as part of the update loop:

Player Translation:

The player ship is translated horizontally based on keyboard input. The moveLeft and moveRight flags set by GLUT keyboard callbacks update playerX each frame.

```
if (moveLeft  && playerX > 30)         playerX -= playerSpeed;  // dx = -6
if (moveRight && playerX < WIN_W - 30) playerX += playerSpeed;  // dx = +6
```

.

Bullet Translation:
Each active bullet is translated upward by 12 pixels per frame, implementing vertical translation: y' = y + 12.

```
bullets[i].y += 12.0f;
```

Enemy (Asteroid) Translation:

Each active asteroid is translated downward at its individual speed, implementing vertical translation: y' = y - speed.

```
enemies[i].y -= enemies[i].speed;
```

Parallax Star Translation:

Each star is translated downward at its own speed (0.3 to 1.2 pixels per frame), creating a multi-layer parallax scrolling effect that gives the illusion of depth and forward motion through space.

```
stars[i].y -= stars[i].speed;
```

Particle Translation with Gravity:

Each particle is translated by its velocity vector each frame, with the vertical velocity reduced by a gravity constant of 0.05 pixels per frame squared, simulating a projectile arc for explosion fragments.

```
particles[i].x  += particles[i].dx;
particles[i].y  += particles[i].dy;
particles[i].dy -= 0.05f;  // simulated gravity
```

2.5 Animation Logic

The animation in this project is handled by the GLUT timer function. The timer() callback is registered with glutTimerFunc and fires every 16 milliseconds, providing approximately 60 frames per second. Each call invokes update() and then glutPostRedisplay() to trigger a fresh render. The double buffer mode (GLUT_DOUBLE) ensures that every frame is fully composed off-screen before being swapped to the display, eliminating flickering.

```
void timer(int val) {
   update();
   glutPostRedisplay();
   glutTimerFunc(16, timer, 0);
}
```

.

Chapter 3

Implementation and Results

This chapter explains how the project was built step by step, from initial window setup through entity management to the final interactive game output.

3.1 Implementation

The project was implemented in C using the OpenGL and FreeGLUT libraries. The implementation follows a modular approach where each visual element and game subsystem has its set of functions. Below is a description of the main implementation steps:

Step 1: Window and Projection Setup

The OpenGL window was created with a size of 700x700 pixels. The orthographic projection was set using gluOrtho2D(0, 700, 0, 700) so that the coordinate system aligns with the pixel layout of the game scene. Double buffering and RGB color mode were enabled.

```
glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
glutInitWindowSize(WIN_W, WIN_H);
gluOrtho2D(0, WIN_W, 0, WIN_H);
```

Step 2: Data Structures

Three main structs were defined to represent the game entities: Particle (position, velocity, color, alpha, size, and life); Bullet (position and active flag); and Enemy (position, speed, radius, nameIdx, and active flag). An additional Star struct holds the parallax background points. All entities are stored in fixed-size arrays allocated globally for zero heap allocation during gameplay.

Step 3: Basic Shape Functions

Two core drawing helper functions were created:

- drawCircle(cx, cy, r): Draws a filled circle using GL_TRIANGLE_FAN with 30 segments, used by the midpoint circle principle.
- drawCircleOutline(cx, cy, r): Draws a circle outline using GL_LINE_LOOP for asteroid borders.
- drawRect(x, y, w, h): Draws a filled rectangle using GL_QUADS.
- DrawText24/DrawText18/DrawText12: Renders bitmap text using three different GLUT font sizes.

.

Step 4: Particle System

The particle system is central to the game's visual richness. spawnParticle() searches the global particles array for an inactive slot and initializes it with random velocity, assigned color, randomized size (2.5 to 6.0 pixels), and a life counter (20 to 45 frames). updateParticles() translates each particle by its velocity, applies a downward gravity of 0.05 per frame, fades alpha proportionally to remaining life, and shrinks the size by 3% per frame. spawnExplosion() calls spawnParticle() 40 times with randomized yellow, orange, and white colors. spawnTrail() emits two blue particles per call from the engine nozzle positions.

```
void spawnParticle(float x, float y, float r, float g, float b) {
   // find inactive slot, set position, random dx/dy, color, alpha=1, random size/life
}
void spawnExplosion(float x, float y) {
   for (i = 0; i < 40; i++) { /* random warm color */ spawnParticle(x, y, r, g, b); }
}
```

Step 5: Player Ship Drawing

The player is drawn by the drawPlayer() function using the global playerX and a fixed playerY of 60. The ship consists of a cyan GL_TRIANGLES hull, two darker blue wing triangles, a light-blue cockpit circle, and two cyan engine glow circles. The player's name label, "Raiyan," is rendered below the ship using drawText12() in a yellow color with a one-pixel black drop shadow.

Step 6: Asteroid System

The drawEnemies() function iterates over all active enemies and draws each as a layered asteroid: a brown base circle, a lighter highlight patch, a darker crater, and a thin dark outline. The asteroid radius is stored per entity (randomly spawned between 18 and 28 pixels). Each asteroid also carries a nameIdx that indexes into the global NAMES array, and the assigned name is rendered as a yellow label with a black shadow directly below the asteroid.

Step 7: Collision Detection

updateEnemies() performs circular collision detection between each active bullet and each active enemy. The Euclidean distance between the bullet center and the asteroid center is compared against the sum of the asteroid radius and the bullet half-width (8 pixels). A hit deactivates both the bullet and the asteroid, spawns an explosion, triggers the crash sound, and awards 100 points.

```
float dist = sqrtf(dx*dx + dy*dy);
if (dist < enemies[i].radius + 8) { /* HIT */ }
```

Step 8: Input Callbacks

Horizontal movement is handled through glutSpecialFunc (arrow keys) using a dual-flag system: specialDown() sets moveLeft or Right to 1, and specialUp() resets them to 0. This allows smooth, held-key movement. Bullet firing and game-state transitions are handled in keyDown() via the

.

standard space key. The R key restarts the game from the game-over state.

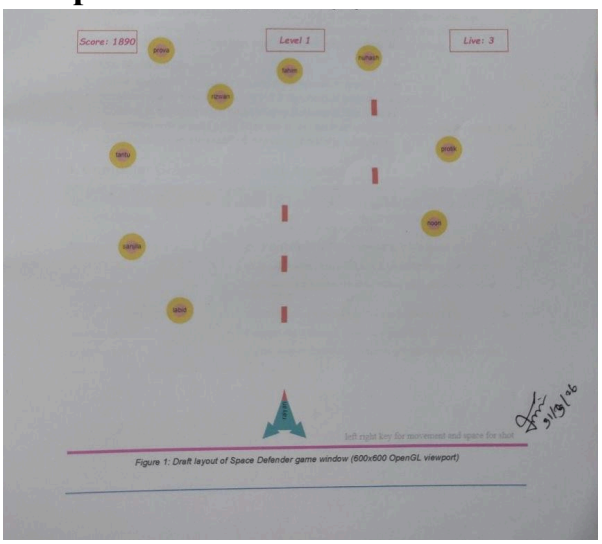
3.1 Output

The output of the project is a 700x700 pixel interactive game window displaying the following elements:

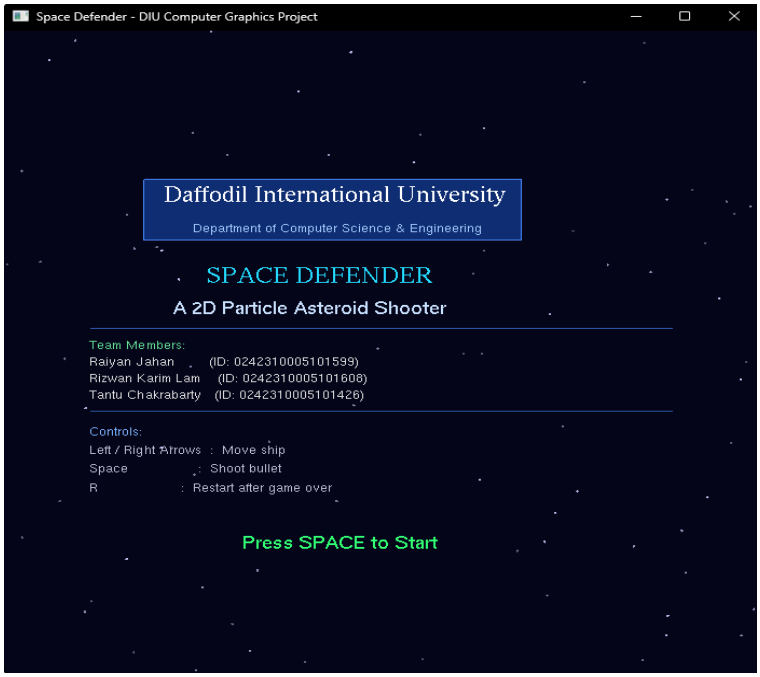
- A styled splash screen with a university branding panel, project title, team member table, control instructions, and a flashing start prompt.
- During gameplay: a deep navy blue background with 80 multi-speed scrolling stars creating parallax depth.
- A triangular cyan player spaceship with wing details, engine glow, and thruster particle trail.
- Up to 12 active asteroids simultaneously, each with layered brown rock texture, crater detail, outline, and a yellow name label.
- Cyan bullet projectiles traveling upward from the ship with glowing white tips.

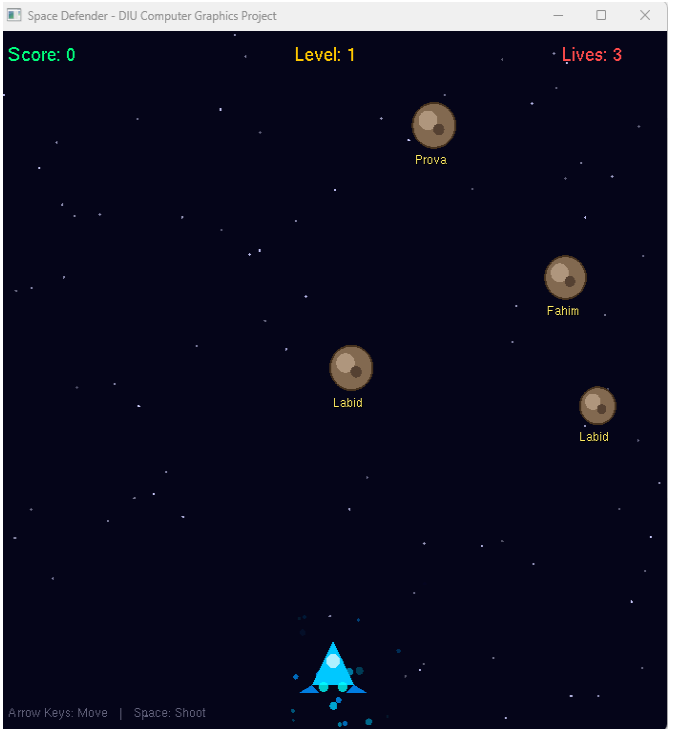
- Colorful explosion particle bursts (up to 40 particles each) appearing at asteroid destruction points.
- AA HUD shows the current score in green (top-left), level in yellow (top-center), and life in red (top-right).
- A semi-transparent game-over overlay showing the final score and restart instructions.

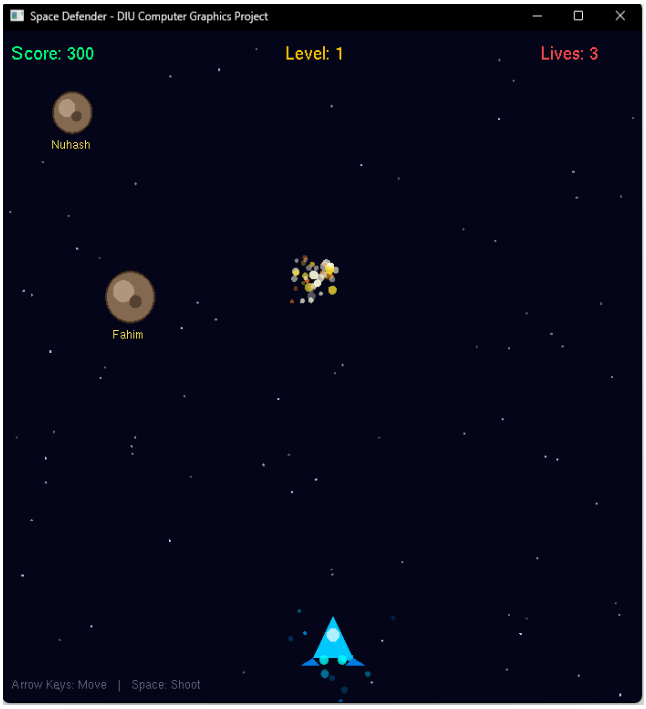
Proposed Model:

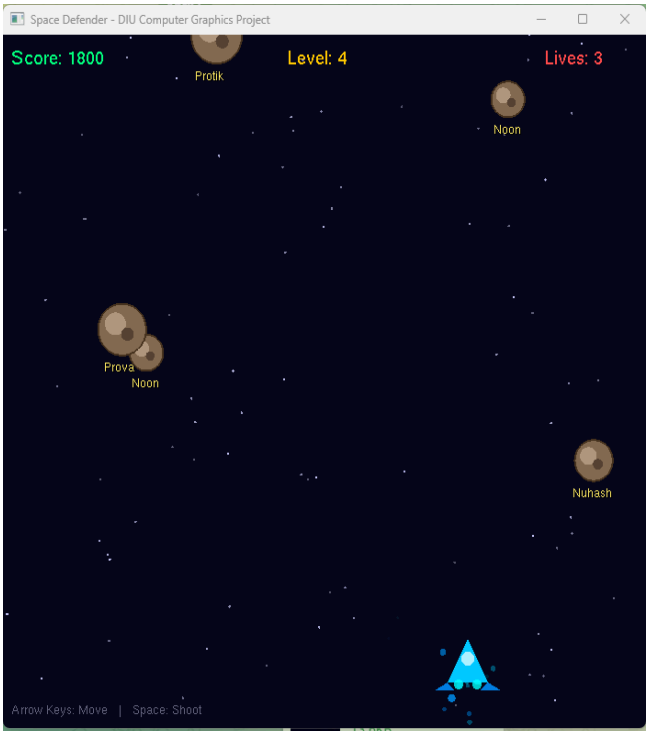


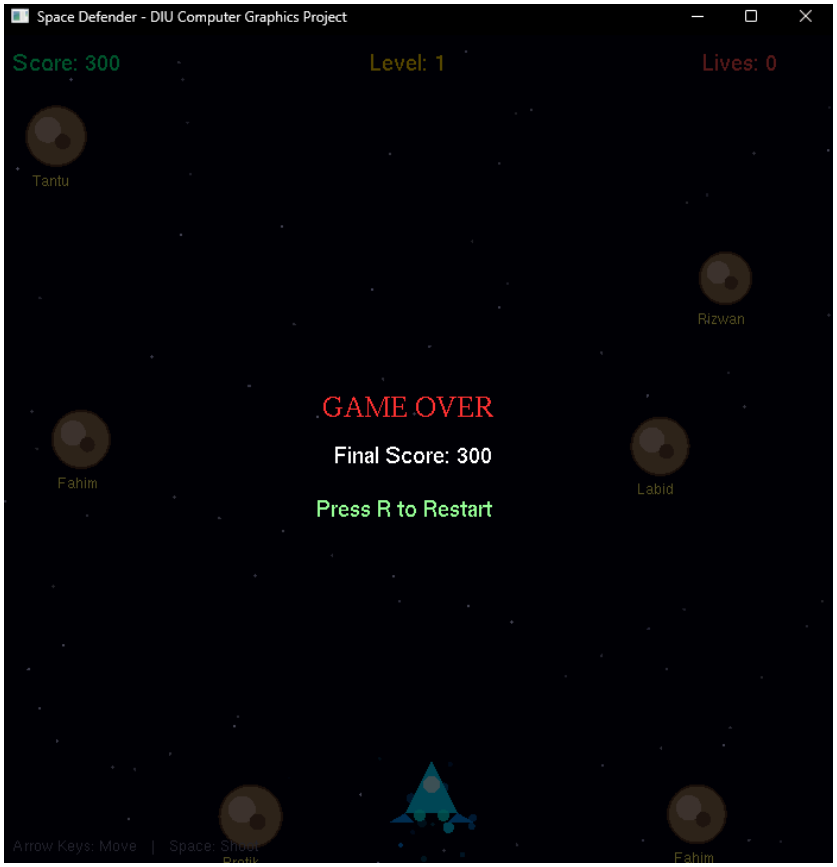
.


Space Defender - DIU Computer Graphics Project
Daffodil International University
Department of Computer Science & Engineering
SPACE DEFENDER
A 2D Particle Asteroid Shooter
Team Members:
Raiyan Jahan (ID: 0242310005101599)
Rizwan Karim Lam (ID: 0242310005101608)
Tantu Chakrabarty (ID: 0242310005101426)
Controls:
Left / Right Arrows : Move ship
Space : Shoot bullet
R : Restart after game over
Press SPACE to Start


Space Defender - DIU Computer Graphics Project
Score: 0
Level: 1
Lives: 3
Prova
Fahim
Labid
Labid
Arrow Keys: Move | Space: Shoot


Space Defender - DIU Computer Graphics Project
Score: 300
Level: 1
Lives: 3
Nuhash
Fahim
Arrow Keys: Move | Space: Shoot


Space Defender - DIU Computer Graphics Project
Score: 1800
Protik
Level: 4
Lives: 3
Noon
Prova
Noon
Nuhash
Arrow Keys: Move | Space: Shoot



3.3 Discussion

The project successfully demonstrates the integration of multiple computer graphics concepts into a single real-time interactive game. All objects were drawn using basic OpenGL primitives: triangles, quads, circles, and lines. The Midpoint Circle Algorithm was used for all circular shapes. The Bresenham's line concept was applied for line-based rendering. 2D translation was applied to every moving entity in the scene, including the player, bullets, asteroids, stars, and particles.

The particle system adds significant visual polish. The combination of fading alpha, shrinking radius, and gravity-affected velocity produces convincing explosion effects and engine trails that would be difficult to achieve with static drawing functions alone. The dynamic asteroid naming system adds a creative and personalized element to the gameplay by assigning each asteroid a name from the team's pool and displaying it as a billboard label.

The use of OpenGL transformation matrix functions (glTranslatef, glPushMatrix, and glPopMatrix) instead of manual coordinate arithmetic can improve one area. Currently, transformation is handled by updating global position variables directly, which is simple and effective but not as scalable as the OpenGL transformation stack for more complex hierarchical objects. Additionally, the collision detection uses simple circle-to-point comparison rather than full bounding-volume hierarchies, which is sufficient for this game scale but would need upgrading for denser scenes.

.

Chapter 4

Engineering Standards and Mapping

This chapter discusses how the project meets engineering standards and maps to the required course and program outcomes.

○ Impact on Society, Environment and Sustainability

Impact on Life
Interactive computer graphics applications like Space Defender demonstrate the foundational principles used in commercial game development, real-time simulation, and digital training systems. The skills developed in building this game, including real-time rendering, entity management, collision detection, and particle effects, are directly applicable to careers in game development, defense simulation, flight training software, and virtual reality applications. Educators widely use educational games built on similar OpenGL techniques to teach spatial reasoning and reaction skills.

- **Impact on Society & Environment**

The game is built entirely using free and open-source software, eliminating proprietary licensing costs and barriers. Real-time graphics applications like this contribute to the broader digital creative economy. By demonstrating that complete interactive software can be built with minimal computational resources (no GPU shader pipeline required), this project also promotes sustainable computing: the game runs efficiently on low-specification hardware, consuming far less energy than modern shader-based rendering pipelines.

- **Ethical Aspects**

The project is built using free and open-source tools, including OpenGL, FreeGLUT, and the C language, which have no licensing cost and are available on all major platforms, including Windows, Linux, and macOS. The code is structured in a modular way, making it reusable and extensible for future enhancements such as adding power-ups, enemy formations, background music, texture mapping, or a high-score leaderboard. The skills and patterns established in this project are directly applicable to sustainable engineering fields that require real-time simulation and visualization

- **Sustainability Plan**

The project is built using free and open-source tools, including OpenGL, FreeGLUT, and the C language, which have no licensing cost and are available on all major platforms, including Windows, Linux, and macOS. The code is structured in a modular way, making it reusable and extensible for future enhancements such as adding power-ups, enemy formations, background music, texture

.

mapping, or a high-score leaderboard. The skills and patterns established in this project are directly applicable to sustainable engineering fields that require real-time simulation and visualization

○ Complex Engineering Problem

■ Mapping of Program Outcome

In this section, provide a map of the problem and a solution with targeted program outcomes (POs).

Table 4.1: Justification of Program Outcomes

POs	Justification of Mapping (Project Perspective)
PO1	This project applies mathematical and computational knowledge to draw and animate all game entities using OpenGL. It uses geometry, coordinate systems, trigonometric functions (circle drawing), Euclidean distance (collision detection), and linear algebra (2D translation vectors) to solve complex real-time visualization and interaction problems in computer graphics.
PO5	The project incorporates modern computer graphics tools and technologies such as OpenGL, FreeGLUT, and C++ programming to ensure the application of contemporary engineering practices and visualization techniques in real-world scenarios.
PO9	The project was completed as a group effort. Team members collaborated on task allocation across subsystems (rendering, game logic, particle systems, and UI/HUD), code integration, testing, and documentation. Each member contributed to different parts of the game and the final report.
PO10	The team prepared a structured technical report, clearly explained the implementation of each game subsystem, and will present the project and its interactive gameplay output through a demonstration and viva.

■ Complex Problem Solving

The table below shows the mapping of this project with Complex Engineering Problem (CEP) attributes defined by the Washington Accord framework. Attributes addressed in this project are marked with a checkmark.

Table 4.2: Mapping with complex problem-solving. The following table presents the mapping of the project with the **Complex Engineering Problem (CEP) attributes defined by the Washington Accord framework**. This project addresses the attributes marked (✓).

Table 4.2: Mapping with complex problem-solving.

EP1 Dept. of Knowledge	**EP2** Range of Conflicting Requirements	**EP3** Depth of Analysis	**EP4** Familiarity of Issues	**EP5** Extent of Applicable Codes	**EP6** Extent Of Stakeholder Involvement	**EP7** Interdependence

.

✓		✓	✓			

EP1: This project requires us to design, implement, and render complex 2D graphical scenes using OpenGL. These tasks involve applying computer graphics algorithms, transformation logic, and animation techniques, enabling us to develop strong analytical and problem-solving skills in visualization and rendering.
EP3: In this project, we analyze object structures, transformation sequences, and animation timing to create smooth and realistic motion. By breaking complex scenes into smaller components and managing their interactions, we strengthen our ability to formulate and analyze complex graphical problems effectively.

EP4: The project operates within the familiar domain of computer graphics algorithms and OpenGL-based rendering. Students apply known techniques such as line drawing algorithms, geometric transformations, and animation logic to construct structured graphical scenes.

4.2.3 Complex Engineering Activities

Table 4.3 presents the mapping of the project with the **Complex Engineering Activities (CEA) attributes**. The attributes relevant to this project are marked (✓).

Table 4.3: Mapping with complex engineering activities.

EA1	EA2	EA3	EA4	EA5
✓		✓		

EA1: In this computer graphics project, we follow systematic methods to design and document all graphical components, algorithms, and transformations. We organize the project structure, rendering logic, and animation workflow clearly in reports and presentations, ensuring that the technical information is logically presented and easily understood by the audience.

EA3: The project demonstrates innovation by designing and implementing visually engaging graphical scenes that combine multiple graphical elements, transformations, and animation sequences. Students creatively construct scenes such as village environments, traffic systems, or animated natural landscapes, where multiple objects interact dynamically. Through the integration of graphics algorithms, transformation techniques, and animation logic, the project encourages students to develop creative solutions and unique visual representations using OpenGL-based graphics programming

.

Chapter 5

Conclusion

5.1 Summary

This project successfully developed a 2D real-time arcade shooter game named Space Defender using OpenGL and FreeGLUT in C. The game features a scrolling parallax starfield, a player-controlled cyan spaceship, dynamically spawned and named asteroids, upward-travelling bullet projectiles, a real-time particle explosion and engine-trail system, and a complete game-state machine with splash, play, and game-over screens. The Midpoint Circle Algorithm was used for all circular shapes. The Bresenham's line concept was applied for straight-line rendering. 2D translation was applied independently to stars, the player, bullets, asteroids, and particles each frame. The game loop runs at approximately 60 FPS via the GLUT timer function and supports increasing difficulty through level progression. The code was organized in a fully modular structure with separate functions for each entity type and game subsystem. The project meets all required complex engineering problem attributes (EP1, EP3, EP4) and complex engineering activity attributes (EA1, EA3).

5.2 Limitation

The current implementation has the following limitations:

- Asteroids travel only in a straight vertical direction and do not follow curved or diagonal paths.
- The player ship can only move horizontally and cannot move vertically or rotate.
- The game is entirely 2D and does not include any 3D perspective, depth of field, or parallax effects beyond the simple scrolling starfield.
- The circle-drawing function uses trigonometric calculation rather than a strict integer-based midpoint circle algorithm, which is slightly less efficient for very large circles.
- No texture mapping or image loading is used; all visuals are constructed from flat-colored geometric primitives.
- There is no persistent high-score system; the score resets every time the game restarts.

5.3 Future Work

Future versions of this project can implement the following improvements:

- Add power-ups that fall from the top of the screen, such as a spread shot, a shield, or a speed boost, collected by flying the ship through them.
- Implement diagonal or curved asteroid trajectories and asteroid splitting on hit for increased gameplay complexity.

.

- Add a high-score leaderboard saved to a text file so that the best scores persist between sessions.
- Introduce different weapon types selectable by the player, such as laser beams, missiles, or area-effect blasts.
- Use glTranslatef() and glPushMatrix()/glPopMatrix() for cleaner and more scalable transformation management in the drawing functions.
- Add background music using a cross-platform audio library such as OpenAL or SDL_mixer, replacing the simple beep sound with a full soundtrack.
- Implement texture mapping to replace flat-colored shapes with sprite images for the ship, asteroids, and background, significantly increasing visual quality

.

References

[1] OpenGL Programming Guide (The Red Book), Addison-Wesley. Available: https://www.opengl.org/archives/resources/code/samples/redbook/

[2] GeeksforGeeks, "OpenGL Tutorials and Computer Graphics Algorithms," [Online]. Available: https://www.geeksforgeeks.org/opengl-tutorials/

[3] FreeGLUT Documentation, [Online]. Available: https://freeglut.sourceforge.net/docs/api.php

[4] D. F. Rogers, Procedural Elements for Computer Graphics, 2nd ed., McGraw-Hill, 1998.

[5] Stack Overflow, "OpenGL game loop using glutTimerFunc," [Online]. Available: https://stackoverflow.com/

[6] Lazy Foo Productions, "Beginning Game Programming," [Online]. Available: https://lazyfoo.net/tutorials/SDL/

[7] Microsoft Docs, "Beep function (utilapiset.h)," [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/api/utilapiset/nf-utilapiset-beep

.